GitHub Actions — Category 6: Self-Hosted Runners & Scaling

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–5 assumed. Kubernetes operational experience assumed for ARC sections. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → Full YAML/Config → Gotchas ⚠ → Connections

---

Table of Contents

- 6.1 Self-Hosted Runner Setup, Labels, and Runner Groups
- 6.2 Actions Runner Controller (ARC) on Kubernetes
- 6.3 Ephemeral Runners vs Persistent Runners — Tradeoffs
- 6.4 Runner Security Isolation — Why Ephemeral Matters
- 6.5 Cost Optimization — Right-Sizing Hosted Runners, Spot Instances
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

6.1 Self-Hosted Runner Setup, Labels, and Runner Groups

What It Is

A self-hosted runner is a machine you own and operate — a VM, bare-metal server, container, or Kubernetes pod — that registers with GitHub and accepts job dispatch from your GitHub organization or repository. You supply the compute; GitHub supplies the orchestration.

Jenkins equivalent: A Jenkins agent/node. The difference: GitHub-hosted runners are managed by GitHub (you never see the machine). Self-hosted runners are your infrastructure, registered with GitHub via a long-poll HTTP connection — you own the compute, the security, the maintenance, and the cost.

Why Self-Hosted Runners Exist

GitHub-hosted runners are excellent defaults but have hard limits that enterprises hit:

Limitation	GitHub-Hosted	Self-Hosted
Hardware	2–16 vCPU, 7–64 GB RAM (fixed tiers)	Any size — 128-core, 1 TB RAM if needed
Network	Public internet egress only	Private VPC, internal services, air-gapped

Limitation	GitHub-Hosted	Self-Hosted
Storage	14 GB SSD, wiped per run	Persistent if desired, any size
OS/arch	Ubuntu, Windows, macOS (x86 only for Linux)	ARM64, GPU, custom OS, kernel versions
Billing	Per-minute charges (included minutes vary by plan)	Your own compute cost only
Compliance	Data leaves your network	Data stays in your VPC
Customization	Preinstalled tools only, locked kernel	Full control: custom packages, kernel params, GPU drivers

How Runner Registration Works Internally

```
Your machine                    GitHub
    |                               |
    |  Register:                    |
    |  ./config.sh --url ...        |
    |  --token <reg-token>          |
    |  --labels linux,gpu,prod  --> |  GitHub stores runner record
    |                               |  Runner appears in Settings UI
    |                               |
    |  Run:                         |
    |  ./run.sh                     |
    |                               |
    |  Long-poll HTTP request:      |
    |  GET /acquireJobs?  --------> |  GitHub checks queue for matching
jobs
    |  (blocks waiting)             |
    |                           <-- |  Job dispatch (runner labels match
job runs-on)
    |                               |
    |  Execute job                  |
    |  POST job log lines       --> |  GitHub stores logs in real-time
    |  POST step results        --> |
    |                               |
    |  Job complete                 |
    |  Long-poll resumes        --> |  Waiting for next job
```

Key architectural fact: The runner initiates ALL connections to GitHub. GitHub never connects to your runner. This means:

- Runners can be in a private VPC with no inbound rules
- Runners need only outbound HTTPS (port 443) to GitHub's endpoints
- No firewall rules needed on the GitHub side

Runner Registration — Step by Step

```
# Step 1: Download the runner binary
mkdir actions-runner && cd actions-runner
# Get latest version from: https://github.com/actions/runner/releases
curl -o actions-runner-linux-x64-2.321.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.321.0/actions-
runner-linux-x64-2.321.0.tar.gz

# Verify checksum
echo "CHECKSUM  actions-runner-linux-x64-2.321.0.tar.gz" | shasum -a 256 -
c
tar xzf ./actions-runner-linux-x64-2.321.0.tar.gz

# Step 2: Get registration token (expires in 1 hour)
# Option A: GitHub UI: Repo Settings -> Actions -> Runners -> New self-
hosted runner
# Option B: API
REG_TOKEN=$(gh api \
  --method POST \
  /repos/my-org/my-repo/actions/runners/registration-token \
  --jq '.token')
# For org-level runner:
REG_TOKEN=$(gh api \
  --method POST \
  /orgs/my-org/actions/runners/registration-token \
  --jq '.token')

# Step 3: Configure the runner
./config.sh \
  --url https://github.com/my-org/my-repo \
  --token $REG_TOKEN \
  --name "prod-runner-01" \
  --labels "self-hosted,linux,x64,production,gpu" \
  --runnergroup "production-runners" \
  --work "_work" \
  --no-default-labels    # Omit to also add: self-hosted, OS, arch labels
automatically

# Step 4: Install and start as a system service
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status

# For ephemeral runner (exits after ONE job): add --ephemeral flag
./config.sh \
  --url https://github.com/my-org \
  --token $REG_TOKEN \
  --name "ephemeral-runner-$(hostname)" \
  --labels "self-hosted,linux,production" \
  --ephemeral    # Runner exits after completing ONE job
```

Labels — The Routing Mechanism

Labels are how GitHub matches a job's `runs-on:` to available runners. They are the ONLY routing mechanism — there's no weighted routing, no round-robin configuration, just label matching.

```
# Default labels added automatically:
# self-hosted   -- always present
# linux         -- OS type (linux | windows | macOS)
# x64           -- architecture (x64 | arm | arm64)

# Custom labels you assign:
# production    -- environment
# gpu           -- hardware capability
# large         -- instance size
# team-payment  -- team ownership
# us-east-1     -- region

# Using labels in workflows:
jobs:
  standard-build:
    runs-on: [self-hosted, linux, x64]

  gpu-training:
    runs-on: [self-hosted, linux, gpu, large]

  production-deploy:
    runs-on: [self-hosted, linux, production]

  arm-build:
    runs-on: [self-hosted, linux, arm64]

  # Mix: GitHub-hosted for CI, self-hosted for deploy
  test:
    runs-on: ubuntu-latest    # GitHub-hosted

  deploy:
    needs: test
    runs-on: [self-hosted, linux, production]   # Self-hosted
```

Label matching semantics: ALL labels in the `runs-on` array must be present on the runner. If a runner has `[self-hosted, linux, x64, production, gpu]`, a job requesting `[self-hosted, linux, production]` matches (subset is sufficient). A job requesting `[self-hosted, windows]` does not match.

Runner Groups — Access Control

Runner groups control which repositories can USE a runner. This is critical for org-level runners shared across repos.

```
# Create a runner group
gh api \
  --method POST \
  /orgs/my-org/actions/runner-groups \
  -f name="production-runners" \
  -f visibility=selected \
  -f allows_public_repositories=false

# List runner groups
gh api /orgs/my-org/actions/runner-groups

# Add runner to a group (by runner ID)
RUNNER_ID=$(gh api /orgs/my-org/actions/runners --jq '.runners[] |
select(.name=="prod-runner-01") | .id')
gh api \
  --method PUT \
  /orgs/my-org/actions/runner-groups/GROUP_ID/runners/$RUNNER_ID

# Grant specific repos access to the group
gh api \
  --method PUT \
  /orgs/my-org/actions/runner-groups/GROUP_ID/repositories \
  -F repository_ids='[REPO_ID_1, REPO_ID_2]'
```

Runner group visibility options:

- `all` — all repos in the org can use runners in this group
- `selected` — only explicitly granted repos
- `private` — only private repos in the org

Org-Level vs Repo-Level Runners

```
Repo-level runner:
  Registered to: specific repo
  Available to: that repo only
  Configured in: Repo Settings -> Actions -> Runners
  Use for: sensitive workloads that should never touch other repos

Org-level runner:
  Registered to: the org
  Available to: repos you grant access via runner groups
  Configured in: Org Settings -> Actions -> Runners
  Use for: shared infrastructure, platform CI, deploy runners
  Advantage: one fleet serves many repos; centrally managed
```

Complete Self-Hosted Runner Setup — Production Hardened

```bash
#!/bin/bash
# setup-runner.sh — production-hardened runner setup script

set -euo pipefail

RUNNER_VERSION="2.321.0"
RUNNER_USER="github-runner"
RUNNER_HOME="/home/${RUNNER_USER}"
ORG_URL="https://github.com/my-org"
RUNNER_LABELS="self-hosted,linux,x64,production"
RUNNER_GROUP="production-runners"

# Create dedicated non-root user
useradd -m -s /bin/bash "$RUNNER_USER"

# Install runner dependencies
apt-get update
apt-get install -y \
  curl \
  jq \
  git \
  docker.io    # If jobs need Docker

# Add runner user to docker group (for Docker-in-Docker workflows)
usermod -aG docker "$RUNNER_USER"

# Download and install runner as runner user
sudo -u "$RUNNER_USER" bash << EOF
cd "$RUNNER_HOME"
mkdir actions-runner && cd actions-runner

curl -sL \

"https://github.com/actions/runner/releases/download/v${RUNNER_VERSION}/ac
tions-runner-linux-x64-${RUNNER_VERSION}.tar.gz" \
  -o runner.tar.gz
tar xzf runner.tar.gz
rm runner.tar.gz

# Get registration token
REG_TOKEN=\$(curl -s -X POST \
  -H "Authorization: Bearer \${GITHUB_REGISTRATION_TOKEN}" \
  -H "Accept: application/vnd.github.v3+json" \
  "https://api.github.com/orgs/my-org/actions/runners/registration-token"
\
  | jq -r '.token')

./config.sh \
  --url "$ORG_URL" \
  --token "\$REG_TOKEN" \
  --name "$(hostname)" \
  --labels "$RUNNER_LABELS" \
  --runnergroup "$RUNNER_GROUP" \
```

```
  --unattended \
  --ephemeral    # Exit after one job -- will be managed by systemd to
restart
EOF

# Install as systemd service
cd "$RUNNER_HOME/actions-runner"
./svc.sh install "$RUNNER_USER"

# Configure systemd to restart after each job (for ephemeral mode)
# ephemeral runner exits after one job; systemd restarts it (picks up
fresh token)
cat > /etc/systemd/system/actions-runner-reregister.service << 'SYSTEMD'
[Unit]
Description=GitHub Actions Runner (auto-reregister)
After=network.target

[Service]
User=github-runner
WorkingDirectory=/home/github-runner/actions-runner
ExecStartPre=/home/github-runner/reregister.sh
ExecStart=/home/github-runner/actions-runner/run.sh
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
SYSTEMD

systemctl daemon-reload
systemctl enable actions-runner-reregister
systemctl start actions-runner-reregister
```

Interview Answer (30-45 seconds)

> "Self-hosted runners are machines you register with GitHub that accept job dispatch via long-poll HTTP. GitHub never connects TO your runner — your runner polls GitHub outbound-only, so they work in private VPCs with no inbound firewall rules. Registration involves downloading the runner binary, configuring it with a one-time token and labels, and running it as a service. Labels are the routing mechanism — `runs-on:` in your workflow must match a subset of the runner's labels. Runner groups add an access control layer: org-level runners can be restricted to specific repos via groups. You'd choose self-hosted over GitHub-hosted when you need private network access, specialized hardware like GPUs or ARM, specific software not in the hosted image, compliance requirements keeping data in your VPC, or cost optimization at scale."

Gotchas ⚠

- **Registration tokens expire in 1 hour.** If your automation takes longer to set up the runner than that, the `config.sh` call fails. Generate the token immediately before running `config.sh`.
- **`self-hosted` label is required in `runs-on`** if you want to ensure you're targeting a self-hosted runner. Without it, a job with `runs-on: linux` might be interpreted differently. Always include `self-hosted` as the first label.
- **Runner binary version must stay within ~3 minor versions of the latest.** GitHub deprecates old runner versions; runs on outdated runners are eventually blocked with an error message. Automate runner updates.
- **Docker socket access on self-hosted runners** means any job can run `docker run --privileged` and break out of isolation. Only self-hosted runners in trusted repo contexts should have Docker socket access.
- **Removing a runner from GitHub UI does NOT stop the runner process.** The process continues polling but receives no jobs. You must also stop the systemd service and remove the runner from the machine.
- **Org-level runners default to allowing public repo access.** Explicitly set `allows_public_repositories: false` on runner groups to prevent public repos from using your private compute.

6.2 Actions Runner Controller (ARC) on Kubernetes ⚠

What It Is

Actions Runner Controller (ARC) is a Kubernetes operator that manages GitHub Actions runners as Kubernetes workloads. Instead of managing VMs or static runner processes, ARC watches your GitHub job queue and automatically creates and destroys runner pods in response to job demand.

This is the most commonly asked self-hosted runner topic in senior/staff interviews because it represents the intersection of GitHub Actions knowledge and Kubernetes operations knowledge.

Jenkins equivalent: Jenkins Kubernetes plugin — spins up agent pods for each build and destroys them when done. ARC is the GitHub Actions equivalent, with the same fundamental model: controller watches for work, creates pods, runs jobs, destroys pods.

Why ARC Exists

Problem with static self-hosted runners:

```
Static runner fleet (e.g., 10 VMs running runner processes):
  Peak load (9 AM): 50 jobs queued, 10 runners busy, 40 waiting
    -> 40 jobs delayed, developers frustrated
  Off-peak (2 AM): 0 jobs, 10 runners idle
    -> Paying for 10 VMs doing nothing

ARC on Kubernetes:
```

```
  Peak load (9 AM): 50 jobs queued -> K8s spawns 50 runner pods
    -> All 50 jobs run in parallel (subject to cluster capacity)
  Off-peak (2 AM): 0 jobs -> 0 pods running
    -> Zero idle runner cost
```

ARC Architecture — Two Generations

Generation 1: RunnerDeployment / HorizontalRunnerAutoscaler (Legacy)

```
GitHub Actions Orchestrator
        |
        |  (webhook or polling)
        v
ARC Controller Pod (in k8s cluster)
        |
        v
Manages: RunnerDeployment
  -> Creates/scales Runner pods
  -> Each Runner pod: full runner binary + Docker-in-Docker sidecar
        |
        v
Runner Pods (one per job):
  [runner container] + [dind container] + [job init container]
```

Generation 2: RunnerScaleSet (Current — recommended)

```
GitHub Actions Orchestrator
        |
        |  (long-poll via ACTIONS_RUNNER_REQUIRE_JOB_CONTAINER)
        v
ARC Scale Set Listener Pod
  (one per RunnerScaleSet resource)
        |
        |  Acquires job messages from GitHub Actions service
        |  Creates runner pods on-demand
        v
Ephemeral Runner Pods (one per job):
  [init container: register runner]
  [runner container: execute job]
  [cleanup: deregister runner]
        |
        v
Runner exits -> pod deleted -> scale set listener creates next pod for
next job
```

ARC Components — What They Are

```
Controller Manager (controller-manager):
  - Kubernetes operator that reconciles RunnerScaleSet resources
  - Watches for RunnerScaleSet CRD instances in any namespace
  - Deploys and manages Listener pods for each scale set
  - Runs as a single Deployment (or HA with leader election)

Listener Pod (one per RunnerScaleSet):
  - Connects to GitHub's webhook forwarding service
  - Receives job assignment messages from GitHub
  - Calls Kubernetes API to create Runner pods
  - Reports runner scale count back to GitHub
  - Stateful: maintains long-lived connection to GitHub

Runner Pod (one per job, ephemeral):
  - init container: downloads runner binary, calls config.sh --ephemeral
  - main container: runs ./run.sh (executes the actual CI job)
  - After job complete: pod status -> Completed, then deleted by
controller
  - Lifecycle: typically 1-30 minutes

AutoScaling:
  - GitHub-native: GitHub tells ARC "I have N jobs waiting for this scale
set"
  - ARC scales up runner pod count to match pending jobs
  - Scale down: pods delete themselves after job completion (ephemeral)
  - No HPA (Horizontal Pod Autoscaler) needed -- GitHub drives scaling
```

Installing ARC — Complete Setup

```
# Prerequisites: kubectl access to cluster, Helm 3, GitHub PAT or App

# Step 1: Install ARC controller via Helm
helm install arc \
  --namespace arc-systems \
  --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set-controller \
  --set replicaCount=2

# Verify controller is running
kubectl get pods -n arc-systems
# NAME                                  READY   STATUS    RESTARTS
# arc-gha-rs-controller-xxx-yyy        1/1     Running   0
```

```yaml
# Step 2: Create GitHub authentication secret
# Option A: Personal Access Token (simpler, less secure)
# PAT needs: repo (for repo-level), admin:org (for org-level)
apiVersion: v1
kind: Secret
metadata:
  name: github-pat-secret
  namespace: arc-runners
type: Opaque
stringData:
  github_token: "ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
---
# Option B: GitHub App (recommended for production)
apiVersion: v1
kind: Secret
metadata:
  name: github-app-secret
  namespace: arc-runners
type: Opaque
stringData:
  github_app_id: "12345"
  github_app_installation_id: "67890"
  github_app_private_key: |
    -----BEGIN RSA PRIVATE KEY-----
    ...
    -----END RSA PRIVATE KEY-----
```

```yaml
# Step 3: Create RunnerScaleSet for org-level runners
# values-arc-runners.yaml
githubConfigUrl: "https://github.com/my-org"
githubConfigSecret: github-app-secret

# Runner configuration
runnerScaleSetName: "ubuntu-k8s-runners"
minRunners: 0        # Scale to zero when idle
maxRunners: 50       # Maximum concurrent runners

# Container mode: kubernetes (runner runs directly in pod)
# vs dind: Docker-in-Docker (runner + docker daemon in same pod)
containerMode:
  type: "kubernetes"   # or "dind"
  kubernetesModeWorkVolumeClaim:
    accessModes: ["ReadWriteOnce"]
    storageClassName: "fast-ssd"
    resources:
      requests:
        storage: 10Gi

# Runner pod template
template:
  spec:
```

```
      initContainers: []
      containers:
        - name: runner
          image: ghcr.io/actions/actions-runner:latest
          # Resources for the runner container itself
          resources:
            requests:
              cpu: 500m
              memory: 512Mi
            limits:
              cpu: "2"
              memory: 4Gi
          # Environment for the runner
          env:
            - name: RUNNER_FEATURE_FLAG_EPHEMERAL
              value: "true"
      # Node selector -- target specific node pool
      nodeSelector:
        purpose: ci-runners
      # Tolerate the CI node pool taint
      tolerations:
        - key: "purpose"
          operator: "Equal"
          value: "ci-runners"
          effect: "NoSchedule"
      # Service account for pods that need K8s API access
      serviceAccountName: arc-runner-sa
```

```
# Step 4: Install the RunnerScaleSet
helm install ubuntu-k8s-runners \
  --namespace arc-runners \
  --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set \
  -f values-arc-runners.yaml

# Verify
kubectl get pods -n arc-runners
# NAME                                    READY   STATUS
# ubuntu-k8s-runners-listener-xxx        1/1     Running
# (runner pods appear only when jobs are dispatched)
```

Multiple Runner Scale Sets — Specialization

```
# Large runner scale set (for resource-intensive builds)
# values-large-runners.yaml
githubConfigUrl: "https://github.com/my-org"
githubConfigSecret: github-app-secret
```

```yaml
  runnerScaleSetName: "large-k8s-runners"
  minRunners: 0
  maxRunners: 20

  template:
    spec:
      containers:
        - name: runner
          image: ghcr.io/actions/actions-runner:latest
          resources:
            requests:
              cpu: "4"
              memory: 8Gi
            limits:
              cpu: "8"
              memory: 16Gi
      nodeSelector:
        node.kubernetes.io/instance-type: "c5.4xlarge"
      tolerations:
        - key: "large-runner"
          operator: "Exists"
          effect: "NoSchedule"
```

```yaml
# GPU runner scale set (for ML workloads)
# values-gpu-runners.yaml
githubConfigUrl: "https://github.com/my-org"
githubConfigSecret: github-app-secret
runnerScaleSetName: "gpu-k8s-runners"
minRunners: 0
maxRunners: 5

template:
  spec:
    containers:
      - name: runner
        image: ghcr.io/my-org/runner-gpu:latest  # Custom image with CUDA
        resources:
          requests:
            cpu: "4"
            memory: 16Gi
            nvidia.com/gpu: "1"
          limits:
            cpu: "8"
            memory: 32Gi
            nvidia.com/gpu: "1"
    nodeSelector:
      accelerator: nvidia-tesla-t4
    tolerations:
      - key: "nvidia.com/gpu"
        operator: "Exists"
```

```
        effect: "NoSchedule"
    runtimeClassName: nvidia
```

Using ARC Runners in Workflows

```yaml
# Workflow using the ARC runner scale sets
name: Build with ARC

on: [push, pull_request]

jobs:
  standard-build:
    # Target the ARC scale set by its runnerScaleSetName
    runs-on: ubuntu-k8s-runners
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make build && make test

  heavy-compile:
    runs-on: large-k8s-runners   # Targets the large scale set
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: cargo build --release   # Rust compile on 8-core runner

  ml-training:
    runs-on: gpu-k8s-runners       # Targets the GPU scale set
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: python train.py --epochs 100
```

Docker-in-Docker Mode vs Kubernetes Mode

```
Kubernetes mode (containerMode.type: kubernetes):
  - Job containers run as SIBLING pods alongside the runner pod
  - Each container: directive in a step creates a separate K8s pod
  - Runner pod orchestrates sibling pods via K8s API
  - Requires: runner SA has permission to create pods in namespace
  - Benefit: true container isolation per step
  - Downside: more K8s RBAC complexity

DinD mode (containerMode.type: dind):
  - Docker daemon runs as sidecar in the runner pod
  - Job containers run inside that Docker daemon
  - Runner pod has a dind sidecar: docker:dind
  - Benefit: simpler, mirrors GitHub-hosted behavior closely
  - Downside: privileged container required for dind
              shared Docker daemon = security concern
```

```
Choosing:
  - Use kubernetes mode for untrusted workloads (better isolation)
  - Use dind mode for trusted internal teams (simpler setup)
  - Use neither for pure shell/binary jobs (runner container only)
```

ARC Scaling Deep Dive

```
GitHub's scaling algorithm (for scale sets):

1. GitHub tracks: pending_jobs for each scale set
2. GitHub sends scale message to Listener: "you need N runners"
3. Listener creates N runner pods (or up to maxRunners)
4. As jobs complete, runner pods self-delete
5. GitHub sends new scale message based on updated queue

Scale-to-zero behavior:
  minRunners: 0 -> All pods deleted when no jobs pending
  First job dispatch: cold start = pod creation time (~30-60 seconds)
  Subsequent jobs: warm if pod already provisioned from previous dispatch

minRunners > 0: pre-warmed pool
  minRunners: 2 -> 2 runner pods always running
  First 2 jobs: no cold start (pod already waiting)
  Jobs 3+: new pods created on demand
  Cost: paying for 2 idle pods always
```

Interview Answer (30-45 seconds)

> "ARC is a Kubernetes operator that manages GitHub Actions runners as ephemeral pods. The current generation (scale sets) uses a Listener pod per scale set that maintains a long-lived connection to GitHub. When GitHub has jobs for that scale set, it sends a scale message to the Listener, which creates runner pods up to `maxRunners`. Each pod registers as an ephemeral runner, executes exactly one job, and then deletes itself. Setting `minRunners: 0` gives you true scale-to-zero — no idle compute when CI is quiet. You install the controller once via Helm, then create RunnerScaleSet resources for each runner profile — standard, large, GPU — each targeting different K8s node pools. In workflows, `runs-on: your-scale-set-name` routes jobs to that scale set. The main operational difference from static runners: you debug runner issues through Kubernetes tooling — `kubectl logs`, pod events — rather than SSH into a VM."

Gotchas ⚠

- **ARC controller and runners should be in separate namespaces.** Controller in `arc-systems`, runners in `arc-runners` (or per-team namespaces). Mixing them creates RBAC confusion and makes upgrades harder.

- **Scale-to-zero cold start is real.** Pod scheduling + image pull + runner registration = 30-90 seconds before the first job runs after a quiet period. For latency-sensitive workflows, set `minRunners: 2` to keep a warm pool.
- **The Listener pod is a single point of failure.** If the Listener pod crashes, no new jobs are dispatched until it recovers. Run the controller with `replicaCount: 2` and use `PodDisruptionBudgets` on the listener.
- **GitHub App auth is strongly preferred over PAT.** PATs are tied to a human account — when that person leaves, all ARC scale sets stop working. GitHub Apps have installation-scoped credentials not tied to a person.
- **Runner pods need sufficient RBAC if using Kubernetes container mode.** The runner service account needs `create`, `get`, `list`, `watch`, `delete` on `pods` and `pods/log` in the runner namespace. Missing these causes cryptic "unable to create container" errors.
- **Image pull time dominates cold start.** Use a node-local image cache or pre-pull the runner image on all CI nodes via a DaemonSet. The `actions-runner` base image is ~300MB; custom images with tools can be 3-10GB.
- **Upgrading ARC requires upgrading both the controller chart and the scale set charts.** They must be compatible versions. Running mismatched versions causes listener registration failures. Always upgrade controller first, then scale sets.

6.3 Ephemeral Runners vs Persistent Runners — Tradeoffs

What It Is

The fundamental architectural choice in self-hosted runner design: does each runner process handle exactly ONE job and then exit (ephemeral), or does a runner process handle multiple jobs over its lifetime (persistent)?

Jenkins equivalent: Disposable agents (created fresh per build, destroyed after) vs traditional agents (long-lived nodes that run many builds sequentially).

The Core Difference

```
PERSISTENT RUNNER:
  Start -> Register -> Job 1 -> Job 2 -> Job 3 -> ... -> Job N ->
(restart)

  State accumulated across jobs:
  - Files left in $GITHUB_WORKSPACE from Job 1 visible to Job 2
  - Environment variables potentially leaked
  - Installed packages from Job 1 present in Job 2
  - Docker images cached locally (benefit AND risk)
  - Background processes from Job 1 still running during Job 2
```

```
EPHEMERAL RUNNER (--ephemeral flag):
  Start -> Register -> Job 1 -> Exit

  New process starts for Job 2:
  - Fresh $GITHUB_WORKSPACE
  - Clean environment
  - No leftover files, processes, or state
  - (With ARC: fresh pod with fresh container filesystem)
```

Persistent Runners — Use Cases and Configuration

```
# Persistent runner: handles multiple jobs sequentially
# DO NOT use --ephemeral flag
./config.sh \
  --url https://github.com/my-org \
  --token $REG_TOKEN \
  --name "persistent-runner-01" \
  --labels "self-hosted,linux,persistent"
# Runner keeps running, picks up next job after current completes
```

Legitimate use cases for persistent runners:

```
1. Warm Docker layer cache between jobs:
   Job 1 builds Docker image, layers cached locally
   Job 2 (same repo, same branch) -- cache hit, 10x faster build
   Ephemeral runner: cache is gone every job
   Persistent runner: cache survives between jobs

2. Warm dependency cache between jobs:
   Similar to Docker layers -- node_modules, ~/.cargo/registry, etc.
   Though: actions/cache is the correct solution (works with ephemeral
too)
   actions/cache uploads/downloads from GitHub -- adds overhead vs local
cache

3. Local tool installation:
   Runner needs a specific commercial tool that can't be installed via
workflow steps
   Installing on the runner VM once beats installing every job
   Ephemeral runners: must install in every pod (or bake into image)
   Persistent runners: tool installed once, available to all jobs

4. Stateful test environments:
   Test framework requires a persistent local database for seed data
   Database is expensive to initialize (30+ minutes)
   Persistent runner keeps the database warm between test runs
   (Usually better to use services containers, but not always feasible)
```

Ephemeral Runners — Why They're the Default Choice

```
# Ephemeral runner: exits after exactly one job
./config.sh \
  --url https://github.com/my-org \
  --token $REG_TOKEN \
  --name "ephemeral-runner-$(date +%s)" \
  --labels "self-hosted,linux" \
  --ephemeral    # THE KEY FLAG
```

Why ephemeral is the better default:

```
Security:
  - No state leakage between jobs from different repos, branches, or PRs
  - A malicious job cannot leave a backdoor for the next job
  - Secrets from Job A cannot be read by Job B
  - (Detailed in 6.4)

Reproducibility:
  - Same job on same code always gets the same environment
  - "Works on my runner" is eliminated
  - Debugging is deterministic -- no "the runner was in a weird state"

Maintenance:
  - No runner disk filling up with accumulated workspace debris
  - No "the runner has been running for 200 days and is acting weird"
  - OS patches applied on next runner refresh (for ARC: next pod creation)

Compliance:
  - Easier to demonstrate to auditors that jobs are isolated
  - No cross-job data residency concerns
```

Lifecycle Comparison Table

Dimension	Ephemeral	Persistent
Jobs per runner instance	1	Many (until restart/crash)
Workspace state	Fresh per job	Accumulates (may be cleaned)
Docker layer cache	Cold every job (unless pre-pulled)	Warm across jobs (same runner)
Dependency cache	Cold (use actions/cache)	Warm (direct filesystem access)
Security isolation	Complete	Partial (relies on cleanup)

Dimension	Ephemeral	Persistent
Reproducibility	High	Lower (state-dependent)
Debugging complexity	Low (no history)	Higher (check previous job state)
Idle resource usage	None (if scale-to-zero)	Always some (runner process alive)
Cold start overhead	Every job	First job only
ARC integration	Natural fit	Requires non-ephemeral mode

Managing Persistent Runner State

If you must use persistent runners, you MUST implement explicit cleanup:

```
# Workflow: explicit cleanup for persistent runners
name: CI (Persistent Runner)

on: [push]

jobs:
  build-and-test:
    runs-on: [self-hosted, linux, persistent]
    steps:
      # Always clean workspace at the start (in case previous job left
debris)
      - name: Clean workspace
        run: |
          # Remove all files in workspace except .git (checkout needs it)
          find . -mindepth 1 -not -path './.git*' -delete 2>/dev/null ||
true

      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - run: make build && make test

      # Always clean workspace at the end too
      - name: Post-job cleanup
        if: always()
        run: |
          # Remove build artifacts that might contain secrets
          rm -rf dist/ .env* *.key *.pem
          # Remove Docker containers left by the job
          docker ps -aq | xargs -r docker rm -f
          # Remove dangling images
          docker image prune -f
```

Runner Restart Strategy for Persistent + Ephemeral Hybrid

```
# Pattern: persistent runner with periodic forced restart
# Combines workspace caching benefit with fresh environment periodically

# systemd unit with runtime limit
cat > /etc/systemd/system/github-runner.service << 'EOF'
[Unit]
Description=GitHub Actions Runner
After=network.target

[Service]
User=github-runner
WorkingDirectory=/home/github-runner/actions-runner
ExecStart=/home/github-runner/actions-runner/run.sh
Restart=always
RestartSec=5
# Force restart after 8 hours regardless of state
RuntimeMaxSec=28800

[Install]
WantedBy=multi-user.target
EOF
```

Interview Answer (30-45 seconds)

> "Ephemeral runners execute exactly one job and then exit — or in ARC's case, the pod is deleted. Persistent runners handle multiple jobs sequentially over their lifetime. Ephemeral is the correct default for security and reproducibility: there's no state leakage between jobs, no accumulated filesystem debris, and no way for a malicious job to backdoor the environment for the next job. The tradeoff is cold start cost per job — Docker layer caches and local dependency caches are cold every time. Persistent runners retain these caches, which can make individual builds significantly faster, but at the cost of isolation. In practice: use ephemeral for all multi-tenant or multi-repo scenarios. Only use persistent runners for single-tenant dedicated runners where the performance benefit clearly outweighs the security consideration, and even then implement explicit workspace cleanup at both the start and end of every job."

Gotchas ⚠

- **`--ephemeral` mode requires the runner to be re-registered after each job.** The runner binary handles this internally with ephemeral mode, but you need a restart mechanism (systemd with `Restart=always`, or ARC managing pod lifecycle). Without restart, one job depletes your runner.
- **GitHub's own cleanup step between jobs on persistent runners is NOT a security boundary.** GitHub cleans `$GITHUB_WORKSPACE` between jobs on the same runner, but it does NOT clean the rest of the filesystem, environment variables set outside the workspace, or processes started by previous jobs.
- **Docker layer cache on persistent runners is a double-edged sword.** A previous job's malicious or compromised Docker image can persist in the local cache and be pulled by subsequent jobs without

a registry auth check. Always `docker pull` explicitly rather than relying on local cache for security-sensitive images.

- **Persistent runners in shared environments are a compliance risk.** If runner A runs a job for the payment team and runner B runs a job for the analytics team on the same persistent runner fleet (round-robin dispatch), cross-team data leakage is possible if cleanup is incomplete.

6.4 Runner Security Isolation — Why Ephemeral Matters

What It Is

Runner security isolation is the set of properties that prevent a job running on a self-hosted runner from affecting other jobs, stealing secrets, or persisting malicious state. It is the critical security argument for ephemeral runners and the primary reason the GitHub documentation explicitly warns against using persistent self-hosted runners on public or multi-tenant repositories.

The Threat Model for Self-Hosted Runners

```
WHO are we protecting against?

Scenario 1: External attacker via fork PR
  - Public repo accepts PRs from anyone
  - Attacker opens PR with malicious workflow step
  - Workflow runs on self-hosted runner
  - Attacker's code runs directly on YOUR infrastructure
  - Can exfiltrate files, credentials, install backdoors

Scenario 2: Internal developer in shared org
  - Developer's PR has a bug that writes sensitive data to workspace
  - Next PR (different developer, maybe different team) runs on same
runner
  - Persistent runner still has previous developer's data in filesystem

Scenario 3: Supply chain attack
  - A third-party action you use is compromised
  - Compromised action modifies runner environment
  - Modification persists to next job (on persistent runner)
  - Next job's secrets are now at risk

Scenario 4: Privilege escalation
  - Job runs as runner user (non-root)
  - Attacker exploits misconfigured Docker socket, sudo, or SUID binary
  - Attacker gains host access
  - All other jobs running on the same host are compromised
```

Why Ephemeral Runners Address the Threat Model

```
Attack: Attacker leaves malicious file in workspace

Persistent runner:
  Job 1 (attacker):  write /home/runner/work/repo/.git/hooks/pre-commit =
malicious.sh
  Job 2 (victim):    git operations trigger pre-commit hook ->
malicious.sh executes
  Result: Attacker's code runs in victim's job

Ephemeral runner + pod:
  Job 1 (attacker):  write file to container filesystem
  Container deleted after job 1 completes
  Job 2 (victim):    fresh container, fresh filesystem, no hooks
  Result: Attack has zero persistence

Attack: Attacker modifies runner binary / installs rootkit

Persistent runner:
  Job 1 modifies /home/runner/actions-runner/run.sh (if writable)
  Modified binary runs all subsequent jobs with attacker code injected

Ephemeral runner (ARC pod):
  Each pod starts from the same immutable container image
  No modification can persist -- pod is read-only (if configured)
  Next pod = pristine image
```

Security Hardening for Self-Hosted Runners

Layer 1: Runner Process Isolation

```
# Run runner as unprivileged user -- never as root
useradd -m -s /bin/bash -G docker github-runner
# Note: adding to 'docker' group grants effective root -- see layer 3

# Restrict workspace permissions
mkdir -p /home/github-runner/work
chown github-runner:github-runner /home/github-runner/work
chmod 700 /home/github-runner/work

# Prevent runner from modifying its own binary
chown root:root /home/github-runner/actions-runner/run.sh
chmod 555 /home/github-runner/actions-runner/run.sh
# Runner user can execute but not modify the binary
```

Layer 2: Network Isolation

```
# Only allow outbound HTTPS to GitHub's IP ranges
# Block all other outbound (prevents secret exfiltration)

# Get GitHub's IP ranges
curl -s https://api.github.com/meta | jq '.actions'

# iptables rules for runner host
iptables -F OUTPUT
iptables -A OUTPUT -d 140.82.112.0/20 -p tcp --dport 443 -j ACCEPT   #
github.com
iptables -A OUTPUT -d 192.30.252.0/22 -p tcp --dport 443 -j ACCEPT   #
github.com
iptables -A OUTPUT -d 185.199.108.0/22 -p tcp --dport 443 -j ACCEPT  #
github CDN
iptables -A OUTPUT -d 20.201.28.151/32 -p tcp --dport 443 -j ACCEPT  #
Actions service
# Allow DNS
iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
# Block everything else
iptables -A OUTPUT -j DROP

# For ARC: implement this via Kubernetes NetworkPolicy
```

Layer 3: Docker Socket Security

```
# The Docker socket problem:
# docker run -v /:/host alpine chroot /host sh
# -> Any job with docker socket access = root on host

# ARC: Do NOT mount the Docker socket into runner pods
# Instead use one of:
# Option A: Docker-in-Docker (privileged container, isolated daemon)
# Option B: Kaniko (rootless Docker builds, no daemon)
# Option C: Buildah (rootless OCI builds)
# Option D: Use Kubernetes container mode (no Docker needed)

# Kaniko for Docker builds without Docker socket:
- name: Build with Kaniko (no Docker socket needed)
  uses: docker/build-push-action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75
  with:
    # Or directly:
    push: false
    # Use kaniko executor
- run: |
    /kaniko/executor \
      --context . \
      --destination ghcr.io/my-org/myapp:${{ github.sha }} \
      --no-push    # or push directly
```

Layer 4: Kubernetes Security Context for ARC

```
# Secure runner pod spec for ARC (add to RunnerScaleSet values)
template:
  spec:
    securityContext:
      runAsNonRoot: true
      runAsUser: 1001          # Non-root UID
      runAsGroup: 1001
      fsGroup: 1001
      seccompProfile:
        type: RuntimeDefault   # Apply seccomp filtering

    containers:
      - name: runner
        image: ghcr.io/actions/actions-runner:latest
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: false  # Runner needs write access to
workspace
          capabilities:
            drop: ["ALL"]      # Drop all Linux capabilities
            add: []            # Don't add any back

    # Prevent access to cloud provider metadata API
    # (GKE, EKS, AKS all expose instance metadata at 169.254.169.254)
    # A compromised job can steal cloud credentials from metadata
    hostNetwork: false
    hostPID: false
    hostIPC: false
```

Layer 5: Kubernetes NetworkPolicy for ARC

```
# Restrict runner pod network access
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: arc-runner-network-policy
  namespace: arc-runners
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/component: runner
  policyTypes:
    - Ingress
    - Egress
  ingress: []    # No inbound traffic to runner pods
```

```
  egress:
    # Allow DNS resolution
    - ports:
        - protocol: UDP
          port: 53

    # Allow HTTPS to GitHub
    - to:
        - ipBlock:
            cidr: 140.82.112.0/20    # github.com
        - ipBlock:
            cidr: 192.30.252.0/22
    - ports:
        - protocol: TCP
          port: 443

    # Allow access to internal registries and artifact stores
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: registry
    - ports:
        - protocol: TCP
          port: 443

    # Block access to cloud metadata API
    # (no egress rule for 169.254.169.254 = blocked by default)
```

Layer 6: OPA/Gatekeeper Policy — Prevent Privileged Runners

```
# OPA ConstraintTemplate to prevent privileged runner pods
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: denyprivilegedrunners
spec:
  crd:
    spec:
      names:
        kind: DenyPrivilegedRunners
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package denyprivilegedrunners
        violation[{"msg": msg}] {
          input.review.object.kind == "Pod"
          input.review.object.metadata.namespace == "arc-runners"
          container := input.review.object.spec.containers[_]
          container.securityContext.privileged == true
          msg := sprintf("Privileged runner pod denied: %v",
[container.name])
```

```
        }
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: DenyPrivilegedRunners
metadata:
  name: deny-privileged-arc-runners
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces: ["arc-runners"]
```

Public Repo Self-Hosted Runner Warning

```
GitHub's official guidance:
"We recommend that you only use self-hosted runners with PRIVATE
repositories.
This is because forks of your public repository can potentially run
dangerous
code on your self-hosted runner machine by creating a pull request that
executes the code in a workflow."

Why it's dangerous:
  Public repo -> anyone can fork
  Fork PR -> triggers pull_request workflow
  pull_request workflow -> runs on self-hosted runner
  Attacker's code -> runs on YOUR infrastructure

Mitigations if you MUST use self-hosted on public repo:
  1. Only use ephemeral runners (each job = fresh environment)
  2. Use ARC with ephemeral pods on a sandboxed K8s cluster
  3. Require PR approval from maintainer before allowing workflow runs
     (Settings -> Actions -> "Require approval for all outside
collaborators")
  4. Separate runner pool for public repos with no access to internal
resources
  5. Run runners in a fully isolated network segment (no VPC access)
```

Interview Answer (30-45 seconds)

> "Runner security isolation matters because self-hosted runners execute arbitrary code submitted through your workflows. The core risk on persistent runners is state persistence across jobs: a malicious job can write files, hooks, or backdoors that execute in subsequent jobs from different users or repos. Ephemeral runners eliminate this by starting each job from a clean state — for ARC pods, that's a fresh container from an immutable image. Additional hardening layers: run the runner process as a non-root user, apply Kubernetes SecurityContext dropping all capabilities, use

> NetworkPolicy to block outbound connections except to GitHub and your own registries, and block cloud provider metadata API access to prevent credential theft via 169.254.169.254. The single most important rule: never use persistent self-hosted runners on public repositories — fork PRs let anyone run arbitrary code on your infrastructure."

Gotchas ⚠

- **Cloud provider IMDS (Instance Metadata Service) at `169.254.169.254` is reachable from any pod/VM by default.** A compromised job can curl this URL and steal the EC2 instance role, GKE node SA token, or AKS managed identity credentials. Block this with NetworkPolicy egress rules or IMDSv2 enforcement (AWS).
- **`docker` group membership == root.** Adding the runner user to the `docker` group so it can run Docker commands grants it effective root access via `docker run --privileged` or volume mounts. Treat Docker socket access as a root escalation.
- **Job-level `container:` directive mounts the Docker socket into the container** if the runner has Docker access. The job's container can escape to the host the same way. Use Kubernetes container mode in ARC to avoid this.
- **Even ephemeral runners share the underlying node.** In ARC, all runner pods for the same scale set run on the same node pool. A container escape (rare but real) breaks isolation at the node level. Use separate node pools with taints/tolerations to isolate sensitive workload runner pods.
- **GitHub Actions audit logs don't include runner-level events** — they show workflow dispatches, not what code actually ran on the runner. Supplement with host-level audit logging (auditd on VMs, Falco on Kubernetes) for full runner activity visibility.

6.5 Cost Optimization — Right-Sizing Hosted Runners, Spot Instances

---

What It Is

Strategies for minimizing CI/CD infrastructure spend while maintaining acceptable performance and reliability. Covers GitHub-hosted runner tier selection, self-hosted cost modeling, and spot/preemptible instance usage for self-hosted runner fleets.

GitHub-Hosted Runner Pricing and Tiers

```
GitHub-hosted runner pricing (as of mid-2025, subject to change):
  Standard (ubuntu-latest, 2 vCPU, 7 GB RAM):
    Linux:   $0.008 / minute
    Windows: $0.016 / minute  (2x Linux)
    macOS:   $0.08  / minute  (10x Linux)

  Larger runners (requires GitHub Team or Enterprise):
```

```
    ubuntu-4-cores  (4 vCPU,  16 GB):  $0.016 / minute
    ubuntu-8-cores  (8 vCPU,  32 GB):  $0.032 / minute
    ubuntu-16-cores (16 vCPU, 64 GB):  $0.064 / minute
    ubuntu-64-cores (64 vCPU, 256 GB): $0.256 / minute

  ARM64 runners (new, cheaper for compatible workloads):
    ubuntu-arm-4-cores (4 vCPU ARM, 16 GB): $0.010 / minute  (~37% cheaper
than x86 4-core)
```

Right-Sizing GitHub-Hosted Runners

```yaml
# STEP 1: Profile your job's resource usage first
# Add to an existing job to measure:
jobs:
  profiled-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Start resource monitoring
        run: |
          # Log CPU and memory every 10 seconds in background
          (while true; do
            echo "$(date +%s) CPU: $(top -bn1 | grep '%Cpu' | awk '{print
$2}')% MEM: $(free -m | awk 'NR==2{print $3}')MB"
            sleep 10
          done) &
          echo $! > /tmp/monitor.pid

      - name: Your actual build steps
        run: make build && make test

      - name: Stop monitoring and show stats
        if: always()
        run: |
          kill $(cat /tmp/monitor.pid) 2>/dev/null || true
          cat /tmp/resource-log.txt 2>/dev/null || echo "No log generated"

# STEP 2: Interpret results and right-size
# Peak CPU < 150% and peak MEM < 4GB -> ubuntu-latest (2-core) is fine
# Peak CPU > 150% or peak MEM 4-7GB -> ubuntu-4-cores
# Peak CPU > 300% or peak MEM > 14GB -> ubuntu-8-cores
```

```yaml
# STEP 3: Use larger runners only where needed
jobs:
  # Standard: pure tests, linting, static analysis (CPU-light, serial)
  lint:
    runs-on: ubuntu-latest        # $0.008/min -- cheapest
```

```
    steps:
      - run: npm run lint

  # Larger: compilation, parallel test execution (CPU-intensive)
  build:
    runs-on: ubuntu-8-cores       # $0.032/min -- 4x the cost
    # But if build takes 5 min on 8-core vs 18 min on 2-core
    # 5 min x $0.032 = $0.16 vs 18 min x $0.008 = $0.144
    # Actually similar cost! But 8-core gives 3.6x faster feedback
    steps:
      - run: make build

  # ARM64: Docker builds for multi-arch (native ARM builds)
  build-arm:
    runs-on: ubuntu-arm-4-cores   # $0.010/min -- cheaper than x86 4-core
($0.016)
    steps:
      - run: docker build --platform linux/arm64 .
```

Self-Hosted Runner Cost Modeling

```
GitHub-hosted cost for 1000 minutes/month of 8-core Linux:
  1000 min x $0.032 = $32/month per developer

Team of 20 developers, each running 1000 CI minutes:
  20 x $32 = $640/month

Self-hosted on EC2 (c5.2xlarge: 8 vCPU, 16 GB):
  On-demand: $0.34/hour = ~$245/month (24x7)
  Reserved (1-year): $0.136/hour = ~$98/month
  Spot: $0.10-0.15/hour = ~$72-108/month

Self-hosted breaks even when:
  On-demand: 245/0.032 = 7,656 minutes/month of usage to break even
  Reserved:  98/0.032  = 3,063 minutes/month
  Spot:      108/0.032 = 3,375 minutes/month (including interruption
overhead)

For a 20-developer team (20,000 min/month on 8-core):
  GitHub-hosted: $640/month
  Self-hosted on-demand: $245/month (62% savings if keeping one instance
busy)
  Self-hosted spot: $108/month (83% savings)
  Self-hosted ARC (scale-to-zero spot): $40-80/month (typical real-world
estimate)
```

Spot/Preemptible Instances for Self-Hosted Runners

Spot instances (AWS) and Preemptible VMs (GCP) offer 60-90% discount in exchange for potential interruption with 2-minute warning. They work well for CI because:

- Jobs are short-lived (minutes to an hour) — low interruption probability
- Interrupted jobs auto-retry on next available runner (if you handle interruption)
- No state loss if you use ephemeral runners (job just restarts fresh)

```
# AWS: EC2 Spot Instance request for runner fleet
# Using Auto Scaling Group with spot instances

cat > runner-launch-template.json << 'EOF'
{
  "LaunchTemplateName": "github-runner-spot",
  "VersionDescription": "GitHub Actions runner on spot",
  "LaunchTemplateData": {
    "InstanceType": "c5.2xlarge",
    "ImageId": "ami-XXXXXXXX",    # Your runner AMI
    "IamInstanceProfile": {
      "Name": "github-runner-profile"
    },
    "UserData": "IyEvYmluL2Jhc2gK...",   # Base64 runner setup script
    "TagSpecifications": [{
      "ResourceType": "instance",
      "Tags": [{"Key": "Purpose", "Value": "github-runner"}]
    }]
  }
}
EOF

aws ec2 create-launch-template --cli-input-json file://runner-launch-
template.json

# Create Auto Scaling Group with spot instances
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name github-runners-spot \
  --launch-template "LaunchTemplateName=github-runner-
spot,Version=$Latest" \
  --min-size 0 \
  --max-size 20 \
  --desired-capacity 0 \
  --mixed-instances-policy '{
    "InstancesDistribution": {
      "OnDemandBaseCapacity": 2,
      "OnDemandPercentageAboveBaseCapacity": 0,
      "SpotAllocationStrategy": "capacity-optimized"
    },
    "LaunchTemplate": {
      "LaunchTemplateSpecification": {
        "LaunchTemplateName": "github-runner-spot"
      },
      "Overrides": [
        {"InstanceType": "c5.2xlarge"},
```

```
        {"InstanceType": "c5a.2xlarge"},
        {"InstanceType": "c5n.2xlarge"},
        {"InstanceType": "m5.2xlarge"}
      ]
    }
  }' \
  --vpc-zone-identifier "subnet-XXXXX,subnet-YYYYY,subnet-ZZZZZ"
```

ARC on Spot/Preemptible Nodes

```
# Kubernetes node pool with spot instances for ARC runners
# AWS EKS managed node group (Terraform example)
resource "aws_eks_node_group" "spot_runners" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "spot-ci-runners"
  node_role_arn   = aws_iam_role.node.arn

  subnet_ids = var.private_subnet_ids

  capacity_type = "SPOT"    # Spot instances

  instance_types = [
    "c5.2xlarge",
    "c5a.2xlarge",
    "c5n.2xlarge",
    "m5.2xlarge",
    "m5a.2xlarge",
  ]

  scaling_config {
    desired_size = 0
    min_size     = 0
    max_size     = 50
  }

  labels = {
    "purpose"          = "ci-runners"
    "capacity-type"    = "spot"
  }

  taint {
    key    = "purpose"
    value  = "ci-runners"
    effect = "NO_SCHEDULE"   # Only CI runner pods schedule here
  }
}
```

```yaml
# ARC RunnerScaleSet configured to use spot node pool
# values-spot-runners.yaml
githubConfigUrl: "https://github.com/my-org"
githubConfigSecret: github-app-secret
runnerScaleSetName: "spot-k8s-runners"
minRunners: 0      # True scale-to-zero -- pay nothing when idle
maxRunners: 50

template:
  spec:
    containers:
      - name: runner
        image: ghcr.io/actions/actions-runner:latest
        resources:
          requests:
            cpu: "1"
            memory: 2Gi
          limits:
            cpu: "4"
            memory: 8Gi
    nodeSelector:
      purpose: ci-runners
      capacity-type: spot
    tolerations:
      - key: "purpose"
        operator: "Equal"
        value: "ci-runners"
        effect: "NoSchedule"
    # Graceful handling of spot interruption
    terminationGracePeriodSeconds: 120   # Give jobs 2 min to wrap up
```

Handling Spot Interruption Gracefully

```bash
# Spot instance termination handler for ARC nodes
# AWS sends a 2-minute warning before interruption via IMDS
# This script, running as a DaemonSet on spot nodes, gracefully drains

# DaemonSet: spot-interrupt-handler
# Runs on all spot nodes
# Polls EC2 instance metadata for termination notice
# When detected: cordon node, drain pods gracefully

kubectl apply -f - << 'EOF'
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: spot-interrupt-handler
  namespace: arc-systems
spec:
```

```
  selector:
    matchLabels:
      app: spot-interrupt-handler
  template:
    metadata:
      labels:
        app: spot-interrupt-handler
    spec:
      serviceAccountName: spot-handler-sa
      tolerations:
        - key: "purpose"
          operator: "Exists"
          effect: "NoSchedule"
      nodeSelector:
        capacity-type: spot
      containers:
        - name: handler
          image: public.ecr.aws/aws-ec2-spot-interruption-handler/aws-
node-termination-handler:latest
          env:
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
            - name: ENABLE_SPOT_INTERRUPTION_DRAINING
              value: "true"
            - name: ENABLE_REBALANCE_DRAINING
              value: "true"
EOF
```

Cost Optimization Strategies — Ranked by Impact

```
1. Cache dependencies properly (actions/cache)
   Impact: Medium-High
   If npm ci takes 3 min -> 45 sec with cache: saves 2.25 min/run
   At $0.008/min, 100 runs/day: saves $0.18/day = $65/year
   Zero infrastructure change required

2. Fail fast in matrix builds (fail-fast: true for unblocking jobs)
   Impact: Low-Medium
   If test fails on one OS, cancel all 8 other running combinations
   Saves up to 8 x remaining-time of cancelled jobs
```

```
3. Use concurrency: cancel-in-progress for PR CI
   Impact: Medium
   Every push cancels previous run -- no more running stale builds
   If avg CI is 10 min and devs push 3 times per PR:
   Without: 30 min billed per PR
   With:    10 min billed per PR (3x saving on CI cost for active PRs)

4. Right-size runners (profile first, then right-size)
   Impact: High
   Over-provisioned team running 4-core for jobs that need 1-core:
   $0.016/min vs $0.008/min = 2x overspend

5. ARC on spot instances (scale-to-zero + spot pricing)
   Impact: Very High for large teams
   GitHub-hosted at scale: $0.032/min (8-core)
   ARC on spot EKS: ~$0.004-0.006/min effective (including K8s overhead)
   5-8x cost reduction at scale

6. macOS to Linux migration where possible
   Impact: Very High for macOS-heavy orgs
   macOS: $0.08/min vs Linux: $0.008/min = 10x difference
   Most non-iOS tasks can run on Linux
   iOS/macOS testing genuinely needs macOS
   Selectively moving CI validation jobs to Linux saves 90%

7. Cache Docker builds in registry (not GHA cache)
   Impact: Medium
   Registry cache persists beyond 10GB GHA limit and 7-day TTL
   Large images (2GB+): GHA cache hit = restore overhead
   Registry cache: pull delta = often just a few layers

8. Reduce workflow run triggers (path filters)
   Impact: Medium
   on: push: paths: ['src/**', 'tests/**']
   Prevents CI running for README edits, documentation changes
```

Path Filters — Zero-Cost Waste Reduction

```yaml
# Run CI only when relevant files change
name: Application CI

on:
  push:
    branches: [main]
    paths:
      - 'src/**'
      - 'tests/**'
      - 'package*.json'
      - 'Dockerfile'
```

```
      - '.github/workflows/ci.yml'   # Include the workflow itself
  pull_request:
    branches: [main]
    paths:
      - 'src/**'
      - 'tests/**'
      - 'package*.json'
      - 'Dockerfile'
      - '.github/workflows/ci.yml'

# Paths to EXCLUDE (docs, READMEs, unrelated configs):
# Note: GitHub uses paths OR paths-ignore, not both simultaneously

on:
  push:
    paths-ignore:
      - '**/*.md'
      - 'docs/**'
      - '.github/dependabot.yml'
      - 'LICENSE'
      - '.gitignore'
```

Cost Observability

```
# GitHub Actions usage API -- per-repo billing breakdown
gh api /repos/my-org/my-repo/actions/cache/usage \
  --jq '{cache_size_gb: (.active_caches_size_in_bytes / 1073741824 |
round)}'

# Org-level billing
gh api /orgs/my-org/settings/billing/actions \
  --jq '{
    total_minutes: .total_minutes_used,
    paid_minutes: .total_paid_minutes_used,
    breakdown: .minutes_used_breakdown
  }'

# Find your most expensive workflows
gh api /repos/my-org/my-repo/actions/runs \
  --paginate \
  --jq '.workflow_runs[] | {
    name: .name,
    created: .created_at,
    updated: .updated_at
  }' | jq -s 'group_by(.name) | map({
    workflow: .[0].name,
    runs: length
  }) | sort_by(-.runs)'
```

Interview Answer (30-45 seconds)

> "Cost optimization in GitHub Actions has four levers. First, caching — properly caching dependencies with `actions/cache` cuts install time 60-80% per run, which directly reduces billable minutes. Second, concurrency cancel-in-progress on PR CI — each new push cancels the previous run, so you never bill for stale builds. Third, right-sizing — profile actual CPU and memory usage before choosing a runner tier; over-provisioned 4-core runners for linting jobs is a common 2x overspend. Fourth, self-hosted on spot instances with scale-to-zero — for teams running hundreds of thousands of CI minutes monthly, ARC on spot K8s nodes achieves 5-8x cost reduction versus GitHub-hosted. The macOS caveat: macOS runners cost 10x Linux; migrate every job that doesn't need macOS to Linux for immediate large savings. Measure before optimizing: the billing API gives per-workflow usage breakdown."

Gotchas ⚠

- **Larger runners are billed from the moment the runner is provisioned**, not from when your first step runs. Runner provisioning takes 15-30 seconds — on large runners at $0.064/min, that's overhead you can't eliminate. For very short jobs (< 2 min), GitHub-hosted standard runner may be more economical than a large runner.
- **Spot interruptions during deploys are catastrophic.** Never run deployment jobs on spot instances. Keep deployment runners on on-demand instances or GitHub-hosted. Reserve spot only for stateless CI (build, test, lint).
- **ARC scale-to-zero cold start has compounding effect.** When a new job arrives after idle period: node scale-up (2-5 min on EKS) + image pull (~1 min for large images) + runner registration (~15 sec) = 3-7 min before job starts. Consider `minRunners: 1` for the latency-sensitive main branch pipeline.
- **Path filters create "missed trigger" confusion.** If a PR changes only `README.md`, the CI workflow doesn't run, and no status check appears on the PR. Branch protection requiring "CI / build" will block the merge even though CI logically "passed" (didn't run). Use always-passing status check bots or GitHub's `paths-ignore` with a corresponding required check bypass for non-code changes.
- **GitHub's billing rounds up to the nearest minute** per job. A 31-second job is billed as 1 minute. Many tiny jobs (linting as a separate job) can cost more in aggregate than one longer combined job, due to rounding. Batch small sequential steps into one job when they're logically related.

Cross-Topic Interview Questions

Q: Design a self-hosted runner architecture for a 200-developer org with mixed workloads: standard CI, GPU ML training, ARM64 Docker builds, and production deployments. Walk through every decision.

> "Four distinct runner profiles, each with its own ARC scale set and node pool. Standard CI on ephemeral pods in a spot node pool — `minRunners: 0`, `maxRunners: 100`, spot c5.2xlarge fleet with capacity-optimized allocation across 5 instance type families for interruption resilience. GPU ML training on a separate scale set targeting a dedicated GPU node pool — p3.2xlarge or g4dn.xlarge,

> on-demand (spot for GPUs is unreliable supply), `maxRunners: 10`, custom runner image with CUDA pre-installed. ARM64 builds on Graviton3 node pool — m7g.2xlarge spot instances, `maxRunners: 20`; native ARM builds run 30-40% faster and cost less than emulating on x86. Production deployments on a SEPARATE on-demand node pool — not spot, not shared with CI, dedicated service account with deploy role permissions, `minRunners: 2` for warm pool, environment protection rules in GitHub enforce approval gates before any pod gets a deploy job.
>
> Authentication: GitHub App (not PAT) for all scale sets — App credentials don't expire when an employee leaves. Network: NetworkPolicy egress rules restricting runner pods to GitHub endpoints and internal registries only. Security: no Docker socket in standard runners (Kaniko for builds), drop all capabilities in SecurityContext, block metadata API access. Monitoring: Falco on all runner nodes for anomalous syscall detection, custom metrics exporter reporting queue depth and scale set utilization to Prometheus."

Q: A developer reports that CI sometimes fails with 'no runner available' and sometimes succeeds. How do you diagnose and fix this?

> "Five possible causes in order of likelihood. One: all runners are busy and the job is queued beyond a GitHub-enforced timeout — check the workflow run's queued time in the UI; if jobs consistently wait more than a few minutes, you need more runners or higher `maxRunners`. Two: label mismatch — the job's `runs-on` array requires a label that no currently available runner has; check runner status in Settings and compare labels carefully. Three: runner group access — the repo doesn't have access to the runner group; verify in Org Settings → Actions → Runner Groups. Four: ARC Listener pod crashed or lost connection to GitHub — `kubectl logs -n arc-runners` on the listener pod; restart the listener pod to re-establish the connection. Five: intermittent cold start timeout — if `minRunners: 0` and node scale-up takes longer than GitHub's job dispatch timeout, the job fails before a runner is available. Increase `minRunners` or pre-warm the cluster. In all cases, the GitHub Actions audit log shows queued timestamps, and `kubectl describe pod` on runner pods shows scheduling and image pull failures."

Q: How does ARC handle the case where a Kubernetes node is terminated mid-job?

> "The behavior depends on whether it's spot interruption or general node failure. For spot interruption with proper setup: the spot interrupt handler DaemonSet detects the termination notice from IMDS (2-minute warning on AWS), cordons the node to prevent new pod scheduling, and begins gracefully draining pods. The runner pod on that node receives SIGTERM via Kubernetes graceful termination. The runner process attempts to complete its current step and mark the job as failed before exiting. GitHub then shows the job as failed with 'runner lost connection.' For ARC RunnerScaleSets, the Listener detects the runner disconnected and GitHub re-queues the job for a new runner — but this retry behavior depends on the event: `push` events do not auto-retry failed runs. The job appears failed, and a developer must re-run manually. Best mitigation: set `terminationGracePeriodSeconds: 120` on runner pods, run deploy jobs only on on-demand nodes (never spot), and use longer step timeouts so in-progress steps have time to complete and fail gracefully rather than getting SIGKILLed."

Quick Reference Cheat Sheet

Runner Registration

```
# Download + configure + run
./config.sh \
  --url https://github.com/ORG          # Org-level or repo URL
  --token $REG_TOKEN \
  --name "runner-name" \
  --labels "self-hosted,linux,x64,custom-label" \
  --runnergroup "group-name" \
  --ephemeral    # Single-job runner

# Install as service
sudo ./svc.sh install && sudo ./svc.sh start
```

Label Routing

```
# ALL labels in array must match runner
runs-on: [self-hosted, linux, production]
# Matches runner with labels: self-hosted,linux,x64,production,us-east-1
# Does NOT match runner with: self-hosted,linux,staging
```

ARC Quick Reference

```
# Controller install
helm install arc --namespace arc-systems --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set-controller

# Scale set install
helm install my-runners --namespace arc-runners --create-namespace \
  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-
set \
  --set githubConfigUrl=https://github.com/my-org \
  --set githubConfigSecret=github-app-secret \
  --set minRunners=0 \
  --set maxRunners=50

# Use in workflow
runs-on: my-runners    # matches runnerScaleSetName
```

Ephemeral vs Persistent Decision

```
Public repo?                  -> MUST use ephemeral
Multi-repo shared runners?    -> Use ephemeral
Performance-critical builds? -> Persistent (with cleanup) or custom image
ARC on K8s?                   -> Always ephemeral (pod per job)
Compliance required?          -> Ephemeral (proves isolation)
```

Cost Decision Tree

```
< 1,000 min/month    -> GitHub-hosted standard (included minutes)
1,000-10,000 min     -> GitHub-hosted (predictable, no ops burden)
10,000-100,000 min   -> Self-hosted on reserved EC2/GKE (2-3x cheaper)
> 100,000 min/month  -> ARC on spot K8s (5-8x cheaper than GH-hosted)

Always:
  Cache dependencies (actions/cache or setup-* built-in)
  concurrency: cancel-in-progress: true for PR CI
  path filters to skip non-code changes
  Never use macOS for Linux-compatible workloads
```

Security Hardening Checklist

```
[ ] Runners as non-root user
[ ] ephemeral flag or ARC (pod per job)
[ ] NetworkPolicy blocking all egress except GitHub + registry
[ ] Block 169.254.169.254 (cloud metadata API)
[ ] No Docker socket unless required; use Kaniko/Buildah instead
[ ] SecurityContext: drop ALL capabilities, runAsNonRoot: true
[ ] OPA policy preventing privileged pods in runner namespace
[ ] NEVER self-hosted persistent runners on public repos
[ ] Runner groups with visibility: selected (not all)
[ ] GitHub App auth (not PAT) for ARC
```

End of Category 6: Self-Hosted Runners & Scaling

> **Next:** Category 7 (Docker, Kubernetes & Cloud Deployments) builds directly on the runner architecture here — OIDC credentials from Category 4 combined with self-hosted runners in a private VPC is the pattern for secure production deployments to air-gapped Kubernetes clusters.
>
> **Priority drill topics from this category:**
>
> - ARC architecture: draw the controller → listener → runner pod flow from memory; explain what happens when a job arrives and when it completes
> - Ephemeral vs persistent: explain the state persistence attack vector on persistent runners; when would you ever choose persistent

> - Spot interruption: explain the 2-minute warning, the DaemonSet handler, and why deploy jobs must never run on spot
> - Cost calculation: walk through the GitHub-hosted vs self-hosted break-even math for a hypothetical team size